
ENGENHARIA DE COMPUTAÇÃO

Disciplina: Engenharia de Software

KA 04 — Construção de Software

(Software Construction)

Guia de Padrões de Codificação e Construção Segura

Instituição de Ensino	UNOESC
Professor(a) Responsável	Odemir Moreira da Mata Jr.
Disciplina	Engenharia de Software
Turma / Semestre	JBA620-6
Integrantes do Grupo	Ana Clara Viganó Prestes de Oliveira, Davi Antônio Basotto Minati, Elian Baldasso Pereira Duarte, Jasmine Masson Alves, Lucas Eduardo Heydt, Vitória Aparecida Vendausen
Data de Entrega	24/06/2026

Histórico de Versões

Versão	Data	Autor(es)	Descrição	Revisado por
v1.0	01/05/2026	Davi Minati	Elaboração do guia de codificação e padrões.	Lucas Heydt
v2.0	24/06/2026	Jasmine Alves	Consolidação final e revisão do documento.	Ana Clara

1.1 Propósito

Este guia estabelece os padrões de construção de software, garantindo a manutenibilidade, a legibilidade e, sobretudo, a segurança e privacidade exigidas pela arquitetura (Privacy by Design)

1.2 Tecnologias Alvo (conforme KA 2)

Backend: Node.js com NestJS (TypeScript).

Frontend: Single Page Application (SPA) com React e TypeScript.

Persistência: PostgreSQL 15.

Comunicação: API RESTful (HTTPS).

2. Convenções de Nomenclatura

2.1 Classes e Interfaces

Exemplo correto (✓): class PatientController, interface CreateAppointmentDto

Exemplo incorreto (✗): class controller_paciente, interface IAppointment (evitar prefixo 'I')

2.2 Métodos e Funções

Exemplo correto (✓): findPatientById(id: string), calculateAppointmentSlots()

Exemplo incorreto (✗): Buscar_Paciente(), get_data_consulta()

2.3 Variáveis e Constantes

As variáveis locais devem seguir o padrão camelCase, iniciando com letra minúscula e utilizando letras maiúsculas para separar palavras subsequentes (ex.: patientName, appointmentDate). Constantes de configuração e valores fixos globais devem utilizar o padrão UPPER_SNAKE_CASE, com todas as letras maiúsculas e palavras separadas por underline (ex.: AES_ENCRYPTION_KEY, MAX_LOGIN_ATTEMPTS).

2.4 Arquivos e Pacotes/Módulos

Os arquivos do projeto devem utilizar o padrão kebab-case, com palavras separadas por hífen e todas as letras minúsculas (ex.: medical-record.service.ts, patient-controller.ts). Esse padrão facilita a organização dos módulos e mantém a consistência na estrutura do sistema.

2.5 Tabelas e Colunas do Banco de Dados

As tabelas e colunas do banco de dados PostgreSQL devem seguir o padrão snake_case, utilizando letras minúsculas e separação por underline (ex.: medical_records, patient_id, appointment_date). Essa convenção melhora a legibilidade das consultas SQL e mantém compatibilidade com as boas práticas de bancos relacionais.

3. Organização do Código-Fonte

3.1 Estrutura de Pacotes / Módulos

src/modules/

Diretório principal que organiza o sistema em módulos independentes, seguindo uma arquitetura modular. Cada módulo concentra suas próprias regras de negócio, controladores, serviços, entidades e repositórios, facilitando a manutenção, escalabilidade e reutilização do código.

patient/

Módulo responsável pelo gerenciamento de pacientes. Implementa operações de cadastro, consulta, atualização e exclusão de dados demográficos, além das validações necessárias para garantir a integridade das informações dos usuários do sistema.

appointment/

Módulo responsável pelo gerenciamento de consultas e agendamentos. Controla a criação, alteração e cancelamento de consultas, além do relacionamento entre pacientes e médicos, verificando disponibilidade de horários e regras de agendamento.

ehr/ (*Electronic Health Record – Prontuário Eletrônico*)

Módulo responsável pelo gerenciamento dos prontuários médicos. Armazena e controla informações clínicas, histórico de consultas, diagnósticos, prescrições, exames e demais registros relacionados ao atendimento do paciente. Por conter dados sensíveis, exige mecanismos adicionais de segurança, controle de acesso e rastreabilidade das operações realizadas.

3.2 Separação por Camadas

Seguir rigorosamente o padrão adotado na arquitetura: **Controller** (interface) —> **Service** (negócio) —> **Repository** (persistência)

3.3 Critério de Complexidade Ciclômática Máxima

Limite de **10** por função. Métodos com alta complexidade devem ser refatorados em sub-funções para facilitar testes e reduzir erros.

4. Padrões de Comentários e Documentação

4.1 Documentação de Classes e Métodos (JavaDoc / JSDoc)

Obrigatório para métodos de serviço e controllers:

/**

- * @param id UUID do paciente
- * @returns Dados criptografados do prontuário
- */

| 4.2 Comentários Inline — quando usar e quando não usar

Usar apenas para explicar "porquês" de decisões complexas de criptografia ou segurança, nunca o "quê" (que o código deve expressar por si só).

5. Tratamento de Exceções e Erros

| 5.1 Padrão de Tratamento de Exceções

O sistema utiliza filtros globais de exceção que interceptam erros não tratados na camada de serviço. Isso garante que a API retorne sempre um objeto JSON padronizado com código de erro amigável, impedindo o vazamento de detalhes técnicos ou estruturais da aplicação (stack trace) que poderiam ser explorados por agentes maliciosos.

| 5.2 Logging de Erros — regras de mascaramento de dados pessoais

Todo erro logado deve passar por um processo de limpeza antes de ser gravado no banco de logs. É uma regra obrigatória de conformidade LGPD que nenhum dado pessoal ou sensível (como nome do paciente, CPF ou diagnóstico) apareça em logs de erro em texto claro; tais informações devem ser mascaradas ou removidas para evitar que o log se torne uma base de dados insegura.

| 6.1 Validação e Sanitização de Entradas

Todas as entradas provenientes do frontend (via DTOs) são submetidas a uma camada de validação rigorosa utilizando class-validator. Esta prática bloqueia injeções de código ou dados malformados antes mesmo de alcançarem a camada de negócio, assegurando que apenas dados estruturados corretamente transitem pelo sistema.

| 6.2 Práticas para Dados Sensíveis de Saúde

O sistema implementa o princípio de Privacy by Design através do Encryption Component. Dados do Prontuário Eletrônico do Paciente (PEP) são obrigatoriamente cifrados com o algoritmo AES-256 antes da persistência no banco de dados. Isso garante que, mesmo que o banco de dados seja acessado fisicamente ou haja vazamento dos arquivos de backup, as informações clínicas permanecerão ilegíveis sem as chaves simétricas protegidas.

6.3 Uso de Bibliotecas Externas

A equipe deve realizar a auditoria periódica de dependências via ferramentas como npm audit antes da homologação de qualquer módulo novo. Bibliotecas externas com vulnerabilidades conhecidas ou mantenedores inativos devem ser evitadas para não comprometer a segurança da API ou introduzir vetores de ataque desconhecidos.

| 6.4 Gerenciamento de Segredos e Credenciais

É terminantemente proibido o armazenamento de chaves de API, credenciais de banco de dados ou chaves de criptografia em código-fonte (hardcoded). Todos os segredos devem ser gerenciados através de variáveis de ambiente seguras ou cofres de segredos (Secret Vaults), assegurando que o código seja agnóstico ao ambiente de infraestrutura.

7. Checklist de Revisão de Código (Code Review)

1. Existe lógica de negócio na Controller? (Deve ser no Service)
2. Dados sensíveis estão sendo logados sem máscara?
3. O princípio de menor privilégio (RBAC) está aplicado?
4. A criptografia está sendo aplicada ao prontuário?
5. Testes unitários cobrem o novo código?

Tabelas e Formulários

| Checklist de Revisão de Código (Code Review)

#	Item de Verificação	Obrigatório?	Observação
1	Lógica de negócio está isolada no Service?	Sim	Evitar lógica pesada na Controller.
2	Dados sensíveis estão sendo mascarados em logs?	Sim	Conformidade com regra LGPD 5.2.
3	O princípio de menor privilégio (RBAC) foi aplicado?	Sim	Validar permissões do endpoint.
4	Criptografia AES-256 aplicada no PEP?	Sim	Dados sensíveis devem ser cifrados.
5	Validação de entrada via DTOs configurada?	Sim	Impedir injeção de dados malformados.
6	Não há segredos (senhas/chaves) hardcoded?	Sim	Utilizar apenas variáveis de ambiente.
7	Complexidade ciclomática está abaixo de 10?	Sim	Refatorar se exceder o limite.
8	Cobertura de testes unitários está adequada?	Sim	Mínimo exigido para o módulo novo.

| Checklist de Construção Segura

#	Prática de Segurança	Aplicada?	Evidência / Como Verificar

| Observações e Referências Consultadas

SWEBOK v4.0 (IEEE, 2024): Capítulo sobre *Software Construction*.
Lei Geral de Proteção de Dados (LGPD), Lei 13.709/2018: Requisitos de segurança e tratamento de dados sensíveis.